SESIÓN N° 08:

Bases de Datos SQLite y JDBC en Java

I
OBJETIVOS

- Conocer los conceptos generales de Bases de Datos Relaciones y de las Clases y Métodos de JDBC en JAVA para manipular los datos de la misma.
- Implementar ejemplo del uso de Clases y Métodos de JDBC en JAVA para manipular los datos de una Base de Datos
- Valorar la importancia de JDBC en Java para manipular una Base de Datos.

II
TEMAS A TRATAR

- Instalación y configuración de conector para SQLite
- Creación y manipulación de Base de Datos con JDBC
- Interfaz PreparedStatement
- Manejo de Transacciones
- Resumen

III
MARCO TEORICO

1. INTRODUCCIÓN AL TEMA

Bases de datos

Una base de datos es una colección organizada de datos hay muchas y diferentes estrategias para organizar datos que nos faciliten el acceso y su manipulación. Un sistema de gestión de base de datos nos brinda mecanismos para almacenar organizar recuperar y modificar los datos de muchos usuarios.

Lenguaje de consulta estructurado

Al día de hoy los sistemas de bases de datos relacionales más populares utilizan el lenguaje llamado SQL. Este lenguaje estándar es usado casi universalmente con todas los las bases de datos relacionales que ejecutan consultas.

Sistemas de gestión de base de datos populares

Algunos de los sistemas de gestión de base de datos más populares son Microsoft SQL Server, Oracle, Sybase, IBMDB 2, Informix, PostreSQL y MySQL.

JDBC

Los programas java interactúan con las bases de datos usando una Interfaz de Programación de Aplicaciones para Conectividad de Base de Datos con Java (JDBC API por sus siglas en inglés). El controlador JDBC permite a las aplicaciones java conectarse a un sistema de gestión de base de datos

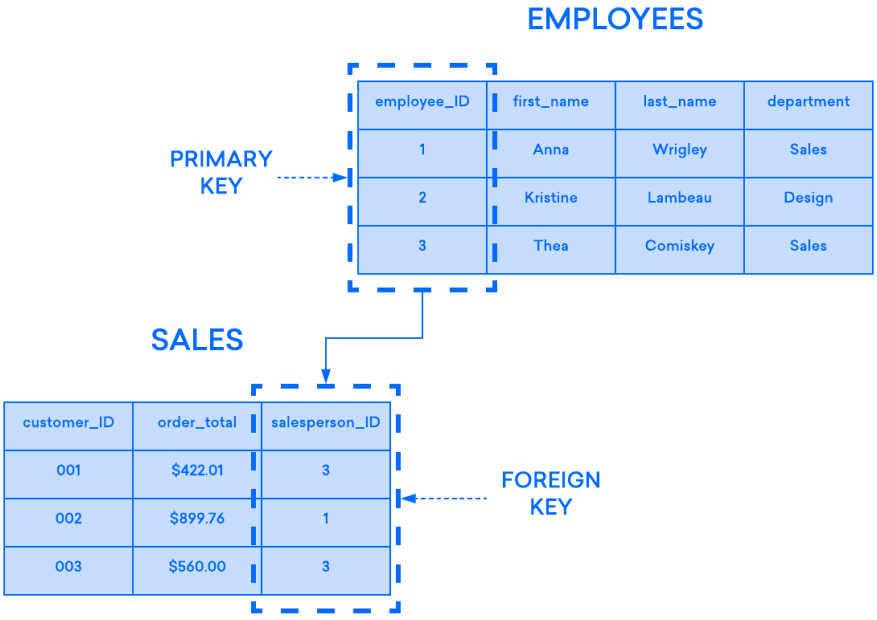
en particular y nos permite manipular la base de datos usando el JDBC API.

2. ¿QUÉ ES UNA BASE DE DATOS RELACIONAL?[1]

Las bases de datos relacionales o también llamadas bases de datos SQL se caracterizan por usar el lenguaje de consulta SQL. Tienen estructuras de datos planas unidas con operaciones de tipo *join* y esquemas rígidos. Están enfocadas en las relaciones entre datos y evitan los datos duplicados.
Las consultas y los índices se modelan después de los datos, que están orientados a filas. También, es posible definir relaciones uno a uno (*one to one*) o uno a varios (*one to many*).
En el modelo relacional definimos tablas que contienen los datos y su estructura. En las consultas es posible devolver varias tablas, relacionando tablas padre con tablas hijo o utilizando operaciones de tipo *join*.

Modelo Entidad-Relación

Las bases de datos relacionales implementan un modelo entidad-relación para representar los datos. Definen entidades, que son los objetos que se quieren guardar en la base de datos. Estas entidades pueden tener tipos que las describen y ocurrencias.
Los atributos son las propiedades o características de las entidades. El atributo identificador permite distinguir una ocurrencia de una entidad unívocamente. Se usan para hacer referencia a un registro de forma individual.
Las relaciones son las asociaciones o vinculaciones entre las entidades. Suelen representar una acción como «pertenece» o «tiene». Las relaciones tienen una cardinalidad, que describe el número de ejemplares de cada entidad que pueden estar relacionados con otra.



Fuente: https://www.digitalocean.com/community/tutorials/understanding-relational-databases-es

Conceptos Básicos de las Bases de Datos Relacionales

A continuación, se describe algunos conceptos y operaciones básicas con las bases de datos

[1] https://aprenderbigdata.com/bases-de-datos-relacionales/

relacionales para entender su funcionamiento y familiarizarnos con sus características.

Operaciones Básicas

Las operaciones básicas de interacción con una base de datos relacional se realizan mediante el lenguaje de consulta SQL. Aquí se tiene unos ejemplos sencillos para ilustrar el funcionamiento.

Creación de una tabla

Podemos crear una tabla con la sentencia CREATE TABLE. Debemos especificar los campos que queremos que tenga. Podemos restringir un campo para que no tenga valores null y establecer claves primarias y ajenas.

```
CREATE TABLE clientes
(
 id char(5) PRIMARY KEY,
 nombre char(40) NOT NULL,
 edad integer,
 direccion char(120),
 cuidad char(25),
 pais char(50),
 email char(100),
 telefono char(15)
);
```

Inserción

Para insertar valores en una tabla existente especificaremos el nombre de la tabla y los valores que queremos insertar a continuación.

```
INSERT INTO clientes
 (id, nombre, edad, direccion, ciudad, pais, email, telefono)
VALUES
 ('1', 'Oscar', 28, 'C/ A', 'Madrid', 'España', NULL, NULL),
 ('2', 'Charlie', 50, 'C/ B', 'Madrid', 'España', NULL, NULL);
```

Actualización

Podemos actualizar los valores que queramos estableciendo condiciones en la sección WHERE.

```
UPDATE clientes
SET edad = 80
WHERE id = 1;
```

Búsqueda

Si queremos obtener los registros de una tabla que cumplen unas determinadas condiciones podemos hacerlo usando un SELECT.

```
SELECT
  id,
  nombre,
  edad
FROM
  clientes
WHERE
  edad > 30
ORDER BY
```

```
10  id DESC;
```

Vistas

En las bases de datos relacionales podemos establecer vistas con SQL. Estas vistas nos proporcionan acceso rápido a datos que se obtendrían de agregar, filtrar o combinar datos de otras tablas.
Las vistas se pueden tratar en SQL como si fueran una tabla una vez que se han creado.

```
1CREATE VIEW clientes_madrid AS
2SELECT id, nombre, edad
3FROM clientes
4WHERE cuidad = 'Madrid';
```

Índices

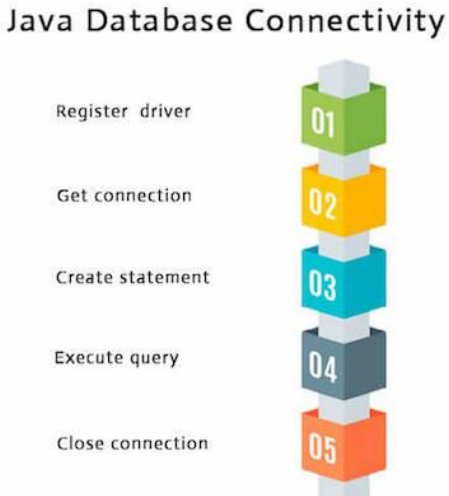
Es posible crear índices sobre determinados campos en las tablas de las bases de datos relacionales. Estos índices son útiles para acelerar las búsquedas que filtran por ese campo.

Transacciones

Una transacción en una base de datos es un conjunto de operaciones que se ejecutan de forma indivisible. El sistema de gestión las considera una unidad y deben realizarse todas las operaciones con éxito (commit) o ninguna.
Las transacciones cumplen las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). En el caso de que existan varias transacciones al mismo tiempo, el sistema puede bloquear o cancelar las que puedan colisionar para evitar inconsistencias.

A. 5 PASOS PARA ESTABLECER CONEXIÓN DE JAVA CON UNA BASE DE DATOS[2]

Hay 5 pasos para conectar cualquier aplicación java con una base de datos usando jl estos pasos son los siguientes:



Fuente: https://www.javatpoint.com/steps-to-connect-to-the-database-in-java

[2] https://www.javatpoint.com/steps-to-connect-to-the-database-in-java

Registrar la clase del controlador

El método *forName*() de la clase *Class* es usado para registrar el controlador de clase. Este método es usado para cargar dinámicamente el controlador de clase.

```
public static void forName(String className) throws ClassNotFoundException
```

Ejemplo asumiendo registro de controlador de BD SQlite:

```
Class.forName("org.sqlite:jdbc");
```

Crear una conexión

El método *getConnection*() de la clase *DriverManager* es usado para establecer conección con la base de datos

```
1) public static Connection getConnection(String url) throws SQLException
2) public static Connection getConnection(String url,String name,String password)  throws SQLException
```

Ejemplo asumiendo conexión con BD Oracle:

```
Connection con = DriverManager.getConnection(
" jdbc:sqlite:C:/sqlite/db/ventas.db ");
```

Crear una sentencia:

El método *createStatement*() de la interfaz *Connection* es usado para crear una sentencia. El objeto de tipo *Statement* (sentencia) es responsable para ejecutar consultas con la base de datos.

```
public Statement createStatement() throws SQLException
```

Ejemplo:

```
Statement stmt = con.createStatement();
```

Ejecutar consultas

El método executeQuery() de la interfaz Statement es usado para ejecutar consultas a la base de datos. Este método retorna un objeto de ResultSet que puede er usado para obtener todos los registros de una tabla.

```
public ResultSet executeQuery(String sql) throws SQLException
```

Ejemplo:

```
ResultSet rs = stmt.executeQuery("select * from clientes");

while(rs.next()){
        System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

Cerrar la conexión

Al cerrar la conexión el objeto Statement y ResultSet se cerrarán automáticamente. El método close() de la interfaz Connection es usado para cerrar la conexión.

```
public void close() throws SQLException
```

Ejemplo:

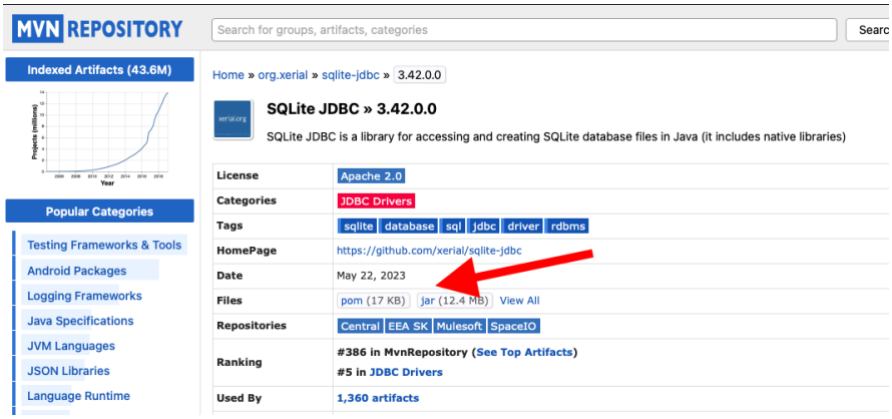
```
con.close();
```

B. INSTALACIÓN Y CONFIGURACIÓN DE CONECTOR SQLITE

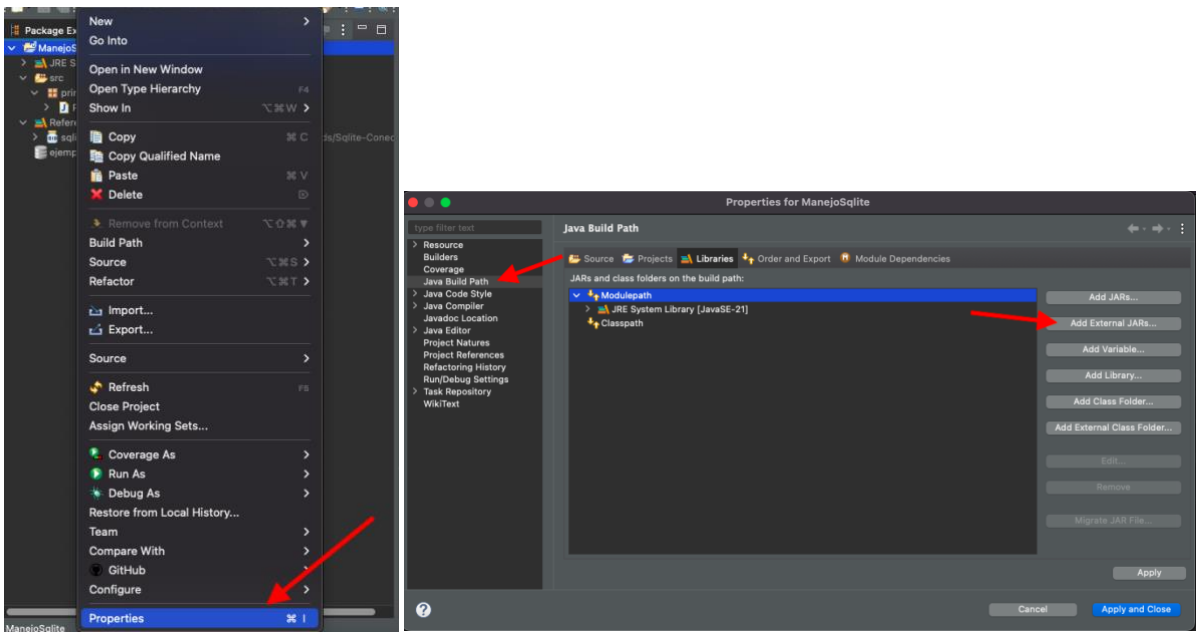
Para conectar una aplicación Java con una base de datos MySQL , es necesario cargar o referenciar el archivo mysqlconnector.jar en la carpeta de librerías .jar de nuestro proyecto en Java.

Podemos descargar este archivo de la siguiente URL

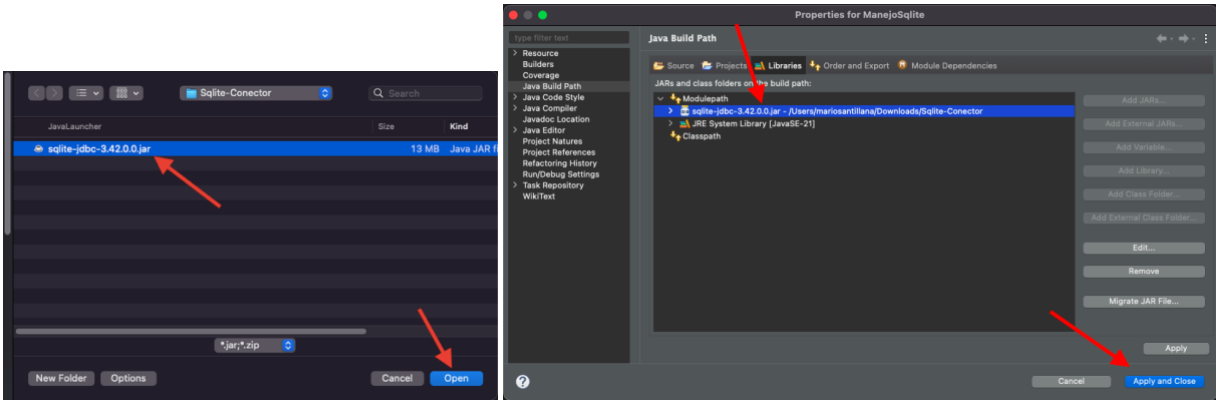
https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc/3.42.0.0



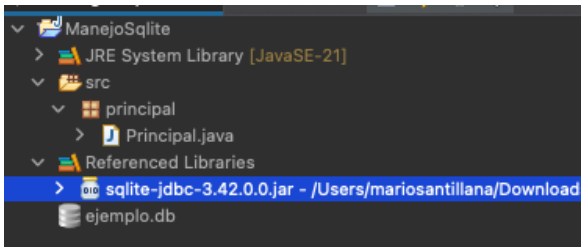
PASO 1: Descargar el archivo .JAR



PASO 2: Agregar el archivo .JAR al proyecto. Parte I



PASO 3: Agregar el archivo .JAR al proyecto. Parte II



PASO 4: Agregar el archivo .JAR al proyecto. Parte III

C. EJEMPLO DE CONEXIÓN DE APLICACIÓN JAVA CON BD SQLITE

Para conectar la aplicación Java con la base de datos SQLite, debemos seguir 5 pasos siguientes.
En este ejemplo estamos usando SQLite como base de datos. Así que necesitamos saber la siguiente información para la base de datos SQLite:

- Clase de controlador: La clase de controlador para la base de datos SQLite es **org.sqlite.JDBC**.
- URL de conexión: La URL de conexión para la base de datos SQLite es **jdbc:sqlite:ejemplo.db** donde *jdbc* es la API, *SQLite* es la base de datos y *ejemplo.db* es el nombre de la base de datos. Podemos usar cualquier base de datos, en tal caso, necesitamos reemplazar el *ejemplo* con el nombre de nuestra base de datos.
- **Primero** crearemos una base datos *ejemplo.db* en caso no exista una con ese nombre, **Segundo** crearemos una tabla. **Tercero** insertaremos 2 registros y **Finalmente** mostraremos estos registros.

```
import java.sql.DriverManager;
import java.sql.ResultSet;s
import java.sql.Statement;
import java.sql.Connection;

public class Principal {

  public static void main(String[] args) {

    try{
      Class.forName("org.sqlite.JDBC");

      Connection con=DriverManager.getConnection(
```

```
            "jdbc:sqlite:ejemplo.db");

        if (con != null) {
            System.out.println("Se creo y/o abrio la base de datos: ");
          }

        Statement stmt = con.createStatement();

        stmt.execute("CREATE TABLE emp (id integer PRIMARY KEY, name text, age integer);");
        System.out.println ("Creación de tabla exitosa");

        stmt.execute("insert into emp VALUES (1,'Carlos',11)");
        stmt.execute("insert into emp VALUES (2,'Bety',22)");
        System.out.println ("Inserción exitosa");

        ResultSet rs = stmt.executeQuery("SELECT * FROM emp");
        while(rs.next())
          System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getInt(3));

        con.close();

      }catch(Exception e){
        System.out.println(e);
      }

    }

}
```

D. LA INTERFAZ PREPAREDSTATEMENT

La interfaz *PreparedStatement* es una subinterfaz de *Statement*. Se utiliza para ejecutar consultas parametrizadas. Veamos el ejemplo de consulta parametrizada:

```
String sql="insert into emp values(?,?,?)";
```

Como se puede ver, estamos pasando el parámetro (?) para los valores. Su valor se establecerá llamando a los métodos setter de *PreparedStatement*. El rendimiento de la aplicación será más rápido si utiliza la interfaz *PreparedStatement* porque la consulta se compila sólo una vez.
El método *prepareStatement*() de la interfaz *Connection* se utiliza para devolver el objeto de *PreparedStatement*. Sintaxis:

```
public PreparedStatement prepareStatement(String query) throws SQLException{}
```

Métodos de interfaz PreparedStatement:

Método	Descripción
public void ***setInt****(int paramIndex, int value)*	establece el valor entero en el índice de parámetros especificado
public void ***setString****(int paramIndex, String value)*	establece el valor String en el índice de parámetro especificado
public void ***setFloat****(int paramIndex, float value)*	establece el valor flotante en el índice de parámetros especificados

public void ***setDouble****(int paramIndex, double value)*	establece el valor Double en el índice de parámetros especificados
public int ***executeUpdate****()*	ejecuta la consulta se utiliza para crear, borrar, insertar, actualizar, eliminar, etc.
public ResultSet ***executeQuery****()*	ejecuta la consulta SELECT. Devuelve una instancia de ResultSet

Ejemplo de interfaz PreparedStatement que inserta el registro

```
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.sql.Connection;

public class Principal {
  public static void main(String[] args) {

    try{
      Class.forName("org.sqlite.JDBC");

      Connection con=DriverManager.getConnection(
          "jdbc:sqlite:ejemplo.db");

      if (con != null) {
          System.out.println("Se creo y/o abrio la base de datos: ");
        }

      PreparedStatement stmt = con.prepareStatement("insert into emp values(?,?,?)");
      stmt.setInt(1,5);
      stmt.setString(2,"Laura");
      stmt.setInt(1,55);

      int i = stmt.executeUpdate();
      System.out.println(i+" records inserted");

      con.close();

    }catch(Exception e){
      System.out.println(e);
    }
  }
}
```

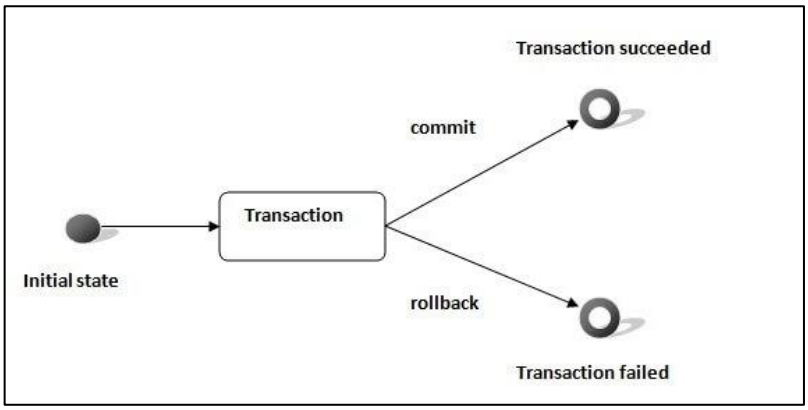
E. MANEJO DE TRANSACCIONES EN JDBC

La transacción representa una sola unidad de trabajo. Las propiedades ACID describen bien la gestión de transacciones. ACID significa *atomicidad*, *consistencia*, *aislamiento* y *durabilidad*.

- Atomicidad significa que todos tienen éxito o ninguno.
- Coherencia garantiza que la base de datos pase de un estado coherente a otro estado coherente.
- Aislamiento garantiza que la transacción esté aislada de otra transacción.

- **Durabilidad** significa que una vez que se ha realizado una transacción, seguirá siéndolo, incluso

La ventaja del manejo de transacciones es que permite un rendimiento rápido porque la base de datos sólo se ve afectada en el momento del *commit* (confirmación).



En JDBC, la interfaz de conexión proporciona métodos para administrar transacciones.

Method	Description
void setAutoCommit(boolean status)	De manera predeterminada es *true* lo que significa que cada transacción es confirmada de manera predeterminada
void commit()	Confirma la transacción
void rollback()	Cancela la transacción

Veamos el ejemplo simple de la administración de transacciones usando Statement.

```
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.sql.Connection;

public class Principal {
  public static void main(String[] args) {

    try{
      Class.forName("org.sqlite.JDBC");

      Connection con=DriverManager.getConnection(
          "jdbc:sqlite:ejemplo.db");

      if (con != null) {
          System.out.println("Se creo y/o abrio la base de datos: ");
        }

      con.setAutoCommit(false);

      Statement stmt=con.createStatement();
      stmt.executeUpdate("insert into emp values(190,'jose',77)");
      stmt.executeUpdate("insert into emp values(191,'diana',88)");
```

```
        con.commit();

        stmt.executeUpdate("insert into emp values(200,'andy',99)");
        con.rollback();

        ResultSet rs = stmt.executeQuery("SELECT * FROM emp");
        while(rs.next())
          System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getInt(3));

        con.close();

      }catch(Exception e){
        System.out.println(e);
      }
    }
}
```

3. RESUMEN

La base de datos SQLite es una base de datos relacional agil que permite realizar operaciones de creación, inserción, actualización, borrado y permite manejar transacciones. Java cuenta con un conector JDBC para poder interactuar con esta base datos. SQLite se utiliza principalmente para aplicaciones donde no se necesitan almacenar una gran cantidad de datos.

IV

(La práctica tiene una duración de 4 horas) ACTIVIDADES

CONTEXTO DEL PROBLEMA

Descripción completa del dominio del problema a resolver en la practica
Conforme un grupo tal como lo indica el docente.
Crear una cuenta de GitHub donde creará y registrará todos sus proyectos:

1. EXPERIENCIA DE PRÁCTICA:

Considerando una base de datos en SQLite creada por usted que contenga una tabla con 3 campos, implementar un ejemplo para los siguiente temas vistos en el Marco Teórico:

- Operaciones Insertar, Recuperar, Actualizar, Borrar registros de una tabla
- Interfaz PreparedStatement
- Manejo de Transacciones en JDBC

V

EJERCICIOS PROPUESTOS

1. Utilizando una Base de Datos en SQLite creada por usted que contenga una tabla con 4 campos. Implementar un aplicación Java que permita mostrar un menú con opciones para poder realizar las operaciones de ingresar, borrar, actualizar y mostrar los datos de la tabla en tiempo de ejecución y por consola. Para poder confirmar las operaciones que modifican los datos de la tabla el usuario deberá ingresar una clave. Si la clave es correcta se deberá confirmar los cambios en la tabla caso contrario se deberá revertirse.

2. Para el ejemplo anterior crea una clase gestora que permita guardar los registros de la tabla en un arreglo de objetos y donde se implementen un método para hacer consultas al arreglo de objetos con las mismas características que se hace con la sentencia SQL pero con la diferencia que la lógica para obtenerse el resultado deberá hacerse desde Java y con las indicaciones que dará el usuario en tiempo de ejecución. La cantidad de parámetros del método deberían considerar:
   a) Qué campos de la tabla deseo mostrar
   b) Si alguno de los campos de registros de la tabla debe cumplir una condición para poder mostrarse
   c) Si los registros deben mostrarse de manera descendente o ascendente en base a un determinado campo.
   d) Si desea limitarse la cantidad de registros a mostrarse

V
CUESTIONARIO

1. ¿Qué es SQLite y cuáles son sus principales características que lo diferencian de otros sistemas de bases de datos?
2. ¿Qué es JDBC y cómo se utiliza para conectarse a una base de datos en Java?
3. Explica el propósito de la clase DriverManager en JDBC.
4. ¿Cómo se utiliza para obtener una conexión a una base de datos y qué parámetros son necesarios para conectarse a SQLite?
5. ¿Qué es una sentencia SQL y cómo puedes ejecutarla en SQLite usando JDBC?
6. Explica la diferencia entre Statement y PreparedStatement en JDBC.
7. ¿Cuándo es recomendable usar PreparedStatement y qué ventajas tiene en términos de seguridad?
8. Describe los pasos para ejecutar una consulta SELECT en una tabla de SQLite y procesar los resultados usando JDBC.
9. ¿Cómo puedes gestionar las transacciones en SQLite usando JDBC?
10. ¿Qué es la función executeUpdate() en JDBC y cuándo se utiliza?
11. Describe cómo puedes manejar errores y excepciones relacionadas con la base de datos cuando trabajas con JDBC.
12. ¿Cómo puedes realizar consultas parametrizadas usando PreparedStatement en JDBC para mejorar el rendimiento y la seguridad?
13. Explica el proceso de cierre de recursos (como Connection, Statement y ResultSet) en JDBC.

¿Por qué es importante cerrar estos recursos correctamente?

VI
BIBLIOGRAFIA Y REFERENCIAS

1. https://aprenderbigdata.com/bases-de-datos-relacionales/
2. https://www.javatpoint.com/steps-to-connect-to-the-database-in-java
3. https://www.sqlitetutorial.net/sqlite-java/

LENGUAJES DE PROGRAMACIÓN III
RUBRICA DE EVALUACIÓN: PRACTICA N° 08: Bases de Datos SQLite y JDBC en Java

% / CRITERIOS	Excelente (100%)	Bueno (75%)	Suficiente (50%)	Insuficiente (0%)
Puntualidad en la presentación del informe (10%)	El informe se entregó en la fecha y hora especificadas.	El informe se entregó con un ligero retraso (menos de 24 horas).	El informe se entregó con un retraso significativo (más de 24 horas).	El informe no se entregó.
Trabajo en equipo (10%)	El equipo demostró una excelente colaboración y comunicación durante todo el proyecto. Todos los miembros contribuyeron de manera equitativa y se apoyaron mutuamente.	El equipo trabajó bien en general, pero hubo algunos problemas menores de comunicación o distribución de tareas.	La colaboración en equipo fue aceptable, pero hubo algunos conflictos o falta de participación de algunos miembros.	El equipo no demostró una buena colaboración. Hubo conflictos significativos o falta de participación de varios miembros.
Completitud del informe (10%)	El informe incluye todos los elementos requeridos y responde a todas las preguntas planteadas en la guía de la práctica.	El informe incluye la mayoría de los elementos requeridos y responde a la mayoría de las preguntas, pero puede faltar algún detalle o explicación.	El informe incluye algunos de los elementos requeridos y responde a algunas preguntas, pero faltan varios detalles importantes o explicaciones.	El informe está incompleto y no responde a las preguntas clave de la guía de la práctica.
Creación y manipulación de Base de Datos con JDBC (30%)	Cumple con todos los siguientes criterios: 1) Se conecta con la Base de Datos 2) Extrae datos 3) Modifica el contenido de las tablas	Cumple con 2 de los siguientes criterios: 1) Se conecta con la Base de Datos 2) Extrae datos 3) Modifica el contenido de las tablas	Cumple con 1 de los siguientes criterios: 1) Se conecta con la Base de Datos 2) Extrae datos 3) Modifica el contenido de las tablas	No Cumple con ninguno de los siguientes criterios: 1) Se conecta con la Base de Datos 2) Extrae datos 3) Modifica el contenido de las tablas
Interfaz PreparedStatement (20%)	Cumple con todos los siguientes criterios: 1) Inserta datos con parámetros 2) Actualiza datos con parámetros 3) Borra datos con parámetros	Cumple con 2 de los siguientes criterios: 1) Inserta datos con parámetros 2) Actualiza datos con parámetros 3) Borra datos con parámetros	Cumple con 1 de los siguientes criterios: 1) Inserta datos con parámetros 2) Actualiza datos con parámetros 3) Borra datos con parámetros	No Cumple con ninguno de los siguientes criterios: 1) Inserta datos con parámetros 2) Actualiza datos con parámetros 3) Borra datos con parámetros
Manejo de Transacciones (20%)	Cumple con todos los siguientes criterios: 1) Activa el manejo de transacciones 2) Manejo adecuado de sentencia Commit 3) Manejo de sentencia Rollback	Cumple con 2 de los siguientes criterios: 1) Activa el manejo de transacciones 2) Manejo adecuado de sentencia Commit 3) Manejo de sentencia Rollback	Cumple con 1 de los siguientes criterios: 1) Activa el manejo de transacciones 2) Manejo adecuado de sentencia Commit 3) Manejo de sentencia Rollback	No Cumple con ninguno de los siguientes criterios: 1) Activa el manejo de transacciones 2) Manejo adecuado de sentencia Commit 3) Manejo de sentencia Rollback